

```
const add = (n1,n2)=>console.log()
```

Closure

- It is function that returns another function that maintains the state of variables declared in the outer function inside the returned function

Currying

- It is function that return another fuunction that accepts single parameter. It works on the SingleResponsibilityPrinciple

<pre>const doWork=()=>{ }</pre>	<pre>const doWork=(e)=>{ }</pre>
<pre>onClick = {()=>{}} onClick - {dowork}</pre>	<pre>onClick = {(e)=>{dowork(e)}}</pre>

```
const doWork=()=>{  
  return ()=>{  
  
}  
}  
  
onClick = {doWork(e)}
```

```
vector<Employee*> empList;  
Employee *e = new Employee()  
empList.push_back(e);  
empList.push_back(new Employee())
```

Function and its types

1. Parameterless function

```
function f1(){  
}
```

2. Parametrized Function

```
function f1(n1){  
}
```

3. Function returning a value

```
function f1(n){  
  return n*n  
}
```

4. Function alias

```
function f1(){  
}  
const f2 = f1;
```

5. Anonymous function

```
const f1 = function(){  
}
```

6. Arrow functions

```
const f1 = ()=>{  
}
```

#Concepts behind the function

1. currying

```
function f1(){  
  return (e)=>{  
  }  
}
```

2. closure(JS) -> (capturing lambda)

```
function f1(){  
  let n1 = 10  
  return (value)=>{  
    return value * n1  
  }  
}
```

3. higher order functions

```
function f1(fn){  
  let n1 = 10  
  fn()  
}  
f1((n1)=>{console.log("Hello")})
```

```
function doWork(table_name,fn_processing){
connect with db;
create sql query;
Select * from table_name
execute the query
get the result
fn_processing(result);
}
```

```
Select * from employee;
Select * from student;
Select * from product;

process the result

get all emps sal > 5000
get all teh student from AC course
products in desc order of price
```

```
doWork(employee,(result)=>{get all emps sal > 5000})
doWork(student,(result)=>{get all the student from AC course})
doWork(product,(result) =>{products in desc order of price})
```

Object in JS

- To create the objects in JS we have 3 ways

- 1. object literal -> {}
- 2. new Object()
- 3. Constructor function

```
int main(){
    struct Date d1;
    struct Date d2;
    struct Date d3;
}

struct Date{
    int day;
    int month;
    int year;
}
```

```
class Employee{
int id;
string name;
double salary;

Employee(int id,string name,double salary){
    this->id = id;
    this->name = name;
    this->salary = salary;
}

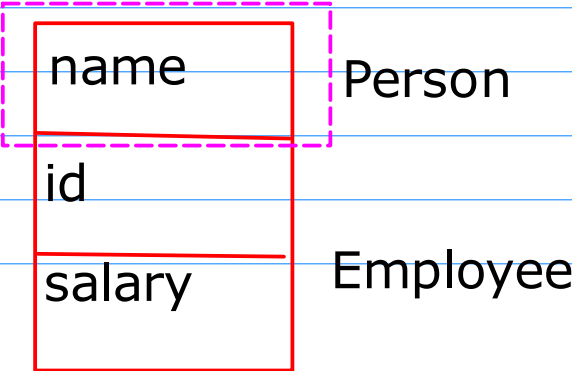
Employee(){
    this->id=0;
    this->name = "";
    this->salary = 0;
}

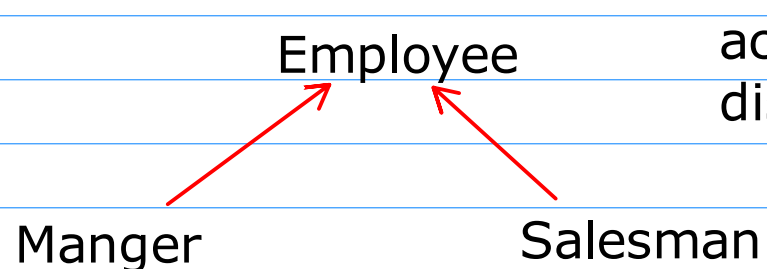
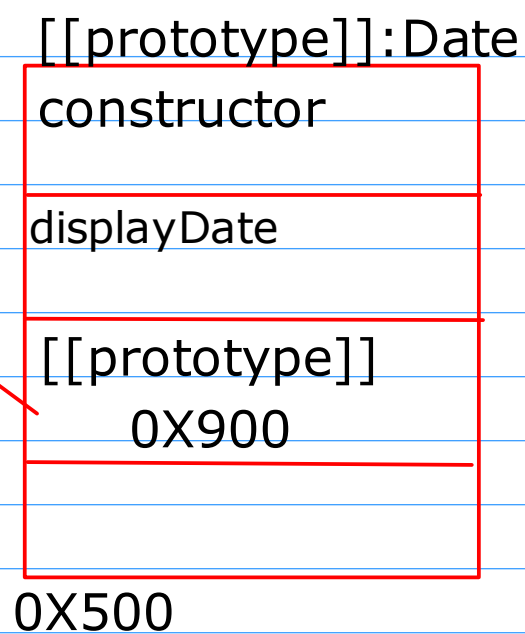
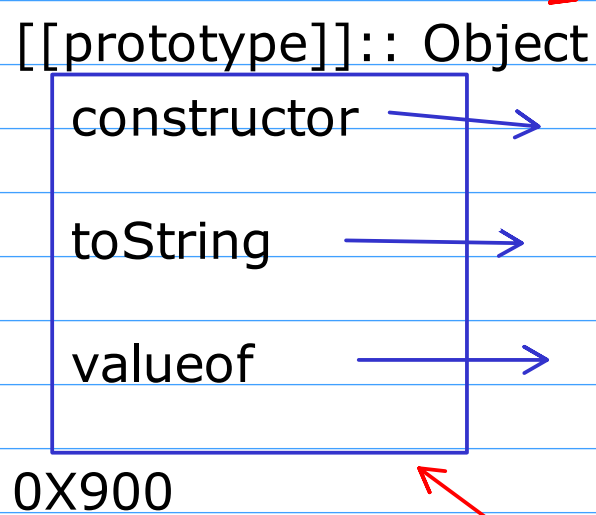
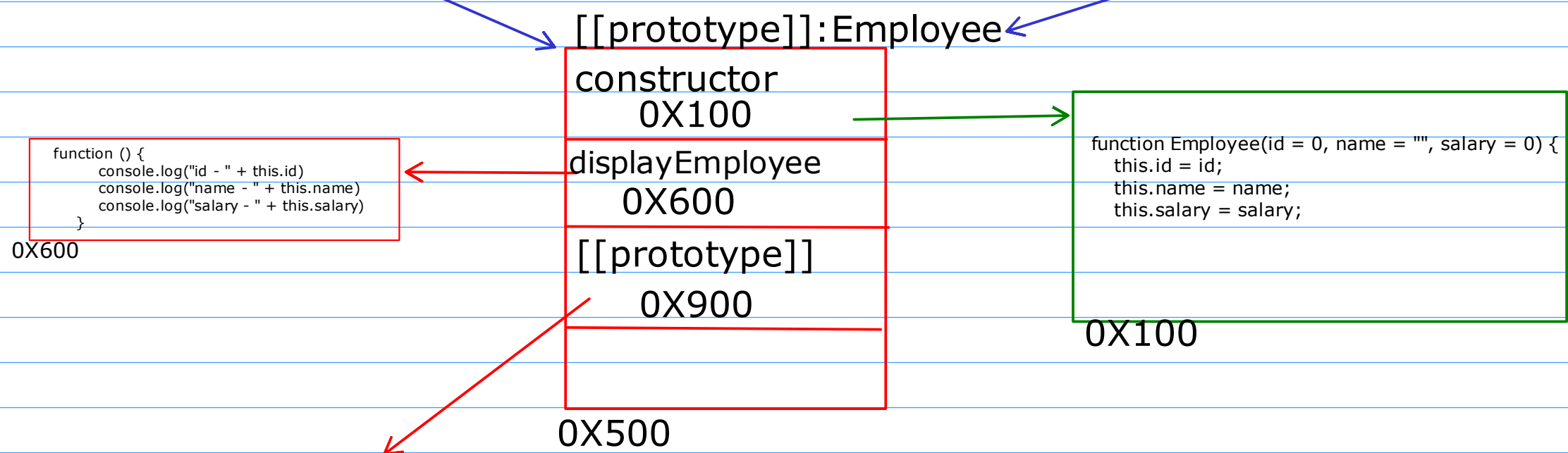
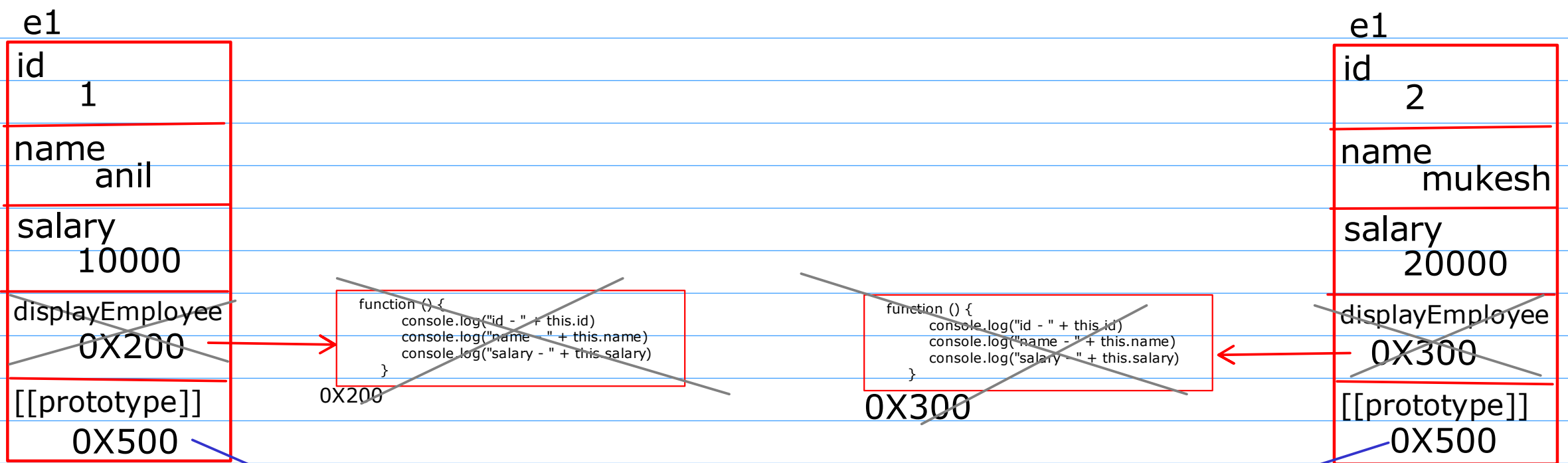
void displayEmployee(){
    cout<<"id - "<<this->id<<endl;
    cout<<"name - "<<this->name<<endl;
    cout<<"salary - "<<this->salary<<endl;
}
}
```

```
int main(){
Employee *e1 = new Employee(1,"anil",10000);
Employee *e2 = new Employee();

e1->displayEmployee();
e2->displayEmployee();
}
```

```
Person *p = new Employee()
```

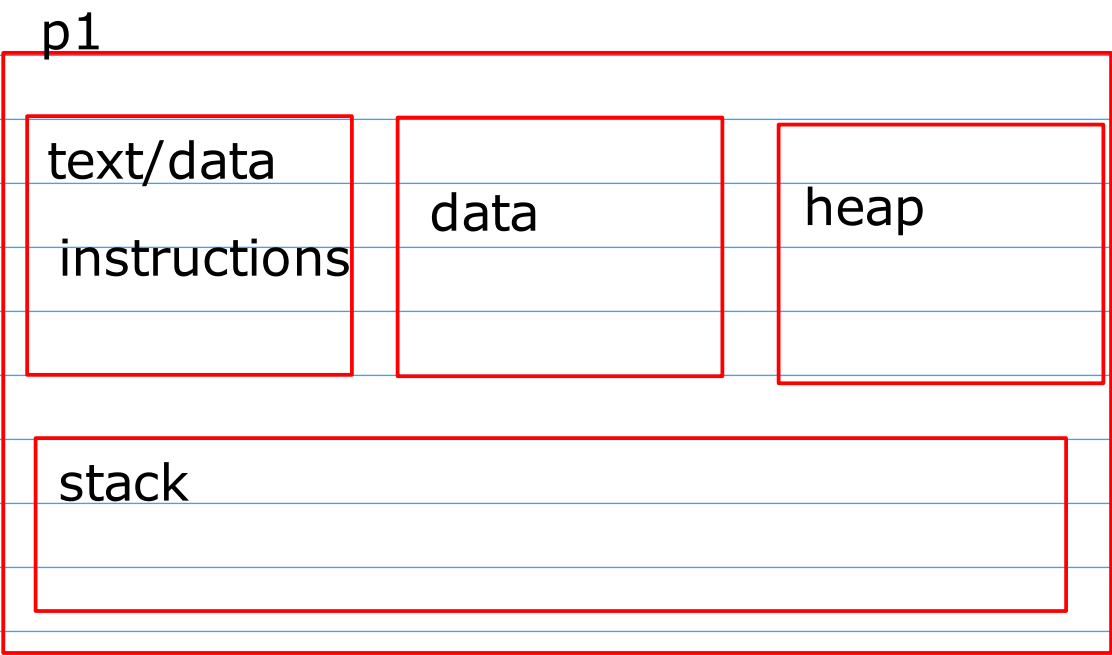




`acceptEmployee()`
`displayEmployee()`

Manager `m1`;
`m1.acceptEmployee()`

Salesman `s1`;
`s1.acceptEmployee()`



```
void add(int n1,int n2){
}
```

```
struct Employee{
void accept()
void display()
}
```

```
struct Date{
void accept()
void display()
}
```

Hirerachy (Reusability)

- Association (has-a relation)
- Inheritance (is-a relation)

```
class Date{
int day,month,year
}
```

```
class Person{
string name,mobile
}
```

```
class Employee : Person{
    Date doj;
}
```

e1

id
name
salary
doj

e2

id
name
salary
doj

e3

id
name
salary
doj

